Espresso Tracker — Frontend Setup Guide

How the React + Vite frontend was built and how it works

1. How Vite + React Was Set Up

Step 1: Project Initialization

```
npm create vite@latest frontend -- --template react
```

This command scaffolded a minimal React project inside the `frontend/` directory. **Vite** is a modern build tool that:

- Serves your React code during development with **Hot Module Replacement (HMR)** — changes appear instantly as you save, no full page reload needed.
- Bundles everything into optimized static files for production deployment (`npm run build` outputs to `dist/`).

Step 2: Installing Dependencies

```
npm install react-router-dom axios
npm install -D sass
```

- **react-router-dom** — Enables client-side routing: navigating between pages (Dashboard → Beans → Add Bean) without full browser page reloads.
- **axios** — HTTP client for making API calls to the Spring Boot backend. Handles JSON parsing and error handling.
- **sass** — Lets us write SCSS instead of plain CSS, giving us variables, nesting, mixins, and other powerful features.

Step 3: Vite Proxy Configuration

The `vite.config.js` file contains a critical piece: a **dev proxy** that forwards API requests to the backend:

```
export default defineConfig({
  plugins: [react()],
  server: {
    port: 5173,
    proxy: {
      "/api": {
        target: "http://localhost:9090",
```

```
        changeOrigin: true,
      },
    },
  },
})
```

Why this is needed: When the frontend (port 5173) makes a request to `/api/v1/beans`, the browser would normally block it due to CORS (Cross-Origin Resource Sharing) since the backend is on a different port (9090). The Vite proxy intercepts these requests and forwards them to the backend, avoiding CORS entirely during development.

> 📌 **How the proxy works:**
> Frontend requests `/api/v1/beans` → Vite dev server intercepts → forwards to `http://localhost:9090/api/v1/beans` → Spring Boot responds → Vite sends response back to browser.

---

2. Project Structure (How Files Are Organized)

```
frontend/src/
├── main.jsx              # Entry point — bootstraps the app
├── App.jsx               # Route definitions (URL → Component)
├── assets/               # Static images (hero.png, icons)
├── components/           # Reusable UI pieces
│   ├── Navbar.jsx        # Navigation bar
│   └── Navbar.scss       # Navbar styles
├── pages/                # Full page views (one per route)
│   ├── Dashboard.jsx     # Home page
│   ├── Dashboard.scss
│   ├── BeanList.jsx      # All beans grid
│   ├── BeanList.scss
│   ├── BeanDetail.jsx    # Single bean + brew logs
│   ├── BeanDetail.scss
│   ├── BeanForm.jsx      # Add/Edit bean (reused)
│   ├── BeanForm.scss
│   ├── BrewLogForm.jsx   # Log a brew
│   └── BrewLogForm.scss
├── services/             # Backend communication
│   └── api.js            # All axios API calls
└── styles/               # Global styling
    ├── _variables.scss   # Design tokens
    └── global.scss       # Base styles & utilities
```

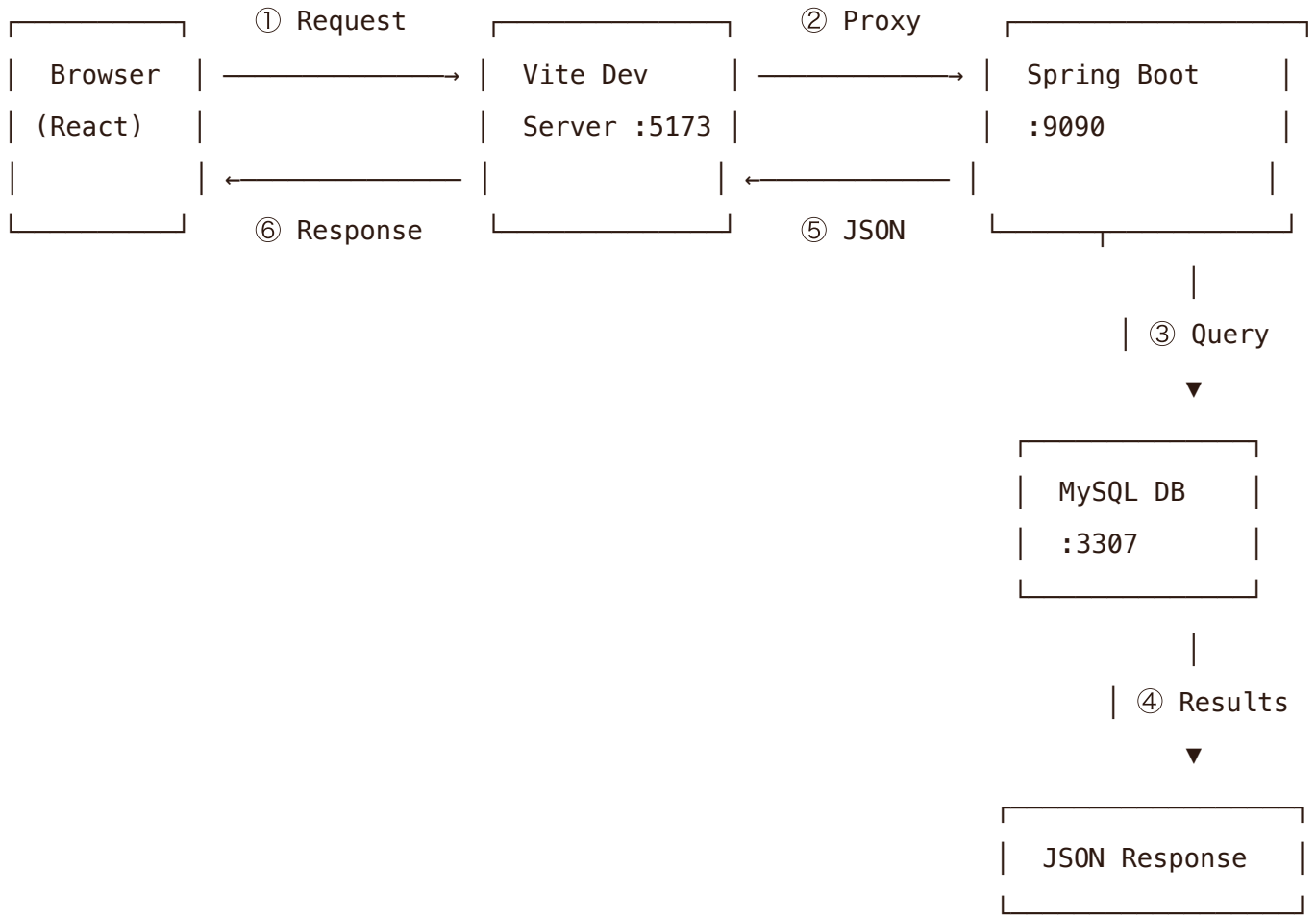
What each folder does:

FOLDER	PURPOSE
`components/`	Reusable pieces used across multiple pages (currently just the Navbar). If we added modals, buttons, or cards as standalone components, they'd go here.
`pages/`	Full page views — each file corresponds to a route. A page can be thought of as "what the user sees at a given URL."
`services/`	Code that talks to external services (our Spring Boot API). Keeps API logic separate from UI logic.
`styles/`	Global SCSS files. `_variables.scss` defines colors/fonts/shadows; `global.scss` imports it and provides base styles, button classes, form

layouts, etc.

3. How Data Flows (Request → Response Cycle)

When a user visits a page, here's the complete cycle of how data moves from the database to the screen:



1. ① **User visits a page** (e.g., `/beans`) → React Router renders the `` component.
2. ② **Component mounts** → The `useEffect` hook runs and calls `getBeans()` from `api.js`.
3. ③ **axios sends GET** → `GET /api/v1/beans` → Vite proxy forwards to `localhost:9090/api/v1/beans`.
4. ④ **Spring Boot handles request** → `BeanController.getAllBeans()` → `BeanService` queries the database via JPA/Hibernate.
5. ⑤ **JSON response returns** → Backend sends back a JSON object with the beans data.

6. ⑥ **React updates state** → `setBeans(data)` triggers a re-render with the new data displayed as cards.

---

4. Routing Map (URL → Component Mapping)

The `App.jsx` file defines all routes using React Router:

```
} />
} />
} />
} />
} />
} />
```

URL	COMPONENT	WHAT IT SHOWS
`/`	Dashboard	Home page with 5 most recent beans + top-rated brews (rating 4-5)
`/beans`	BeanList	All beans displayed as a responsive card grid with Edit/Delete actions
`/beans/new`	BeanForm	Empty form to add a new coffee bean (Roaster, Name, Origin, Roast Level, Tasting Notes)
`/beans/:id`	BeanDetail	Single bean details + table of all brew logs for that bean
`/beans/:id/edit`	BeanForm	Same form, but pre-filled with existing bean data for editing
`/brew-log/new`	BrewLogForm	Form to log a brew: select bean, dose, yield, time, grind, rating, notes

Notice that `BeanForm` is used for **both** adding and editing — it checks if a URL parameter `:id` exists. If yes, it's edit mode and it loads the existing bean data to pre-fill the form. If no, it's create mode with empty fields.

5. API Service Layer (How the Frontend Talks to the Backend)

The `services/api.js` file is the single source of truth for all backend communication. Here's every function it exports:

```
import axios from "axios";

const api = axios.create({
  baseURL: "/api/v1",          // All requests start with /api/v1
  headers: { "Content-Type": "application/json" },
});

// Beans
export const getBeans     = (page, size)  → GET  /beans
export const getBeanById  = (id)          → GET  /beans/{id}
export const createBean   = (data)        → POST /beans
export const updateBean   = (id, data)    → PUT  /beans/{id}
export const deleteBean   = (id)          → DELETE /beans/{id}

// Brew Logs
export const getLogsByBeanId  = (beanId)  → GET  /brew-logs/bean/{beanId}
export const getTopRatedLogs  = ()        → GET  /brew-logs/top-rated
export const createBrewLog    = (data)    → POST /brew-logs
```

Each function returns a Promise that resolves to the JSON response data. Components call these functions and use `async/await` to handle the results.

---

6. Styling System (SCSS Architecture)

Design Tokens (`_variables.scss`)

```
$color-primary:   #4a3728    // Deep brown — navbar, headings
$color-accent:    #d4824a    // Warm orange — buttons, links
$color-background:#faf7f2    // Cream — page background
$color-surface:   #ffffff    // White — cards
$color-text:      #2c1810    // Near-black — body text
$color-border:    #e8ddd0    // Light tan — borders
```

What the global styles provide

- **Base reset** — Consistent box-sizing, removes default margins
- **Typography** — System font stack, proper line-height
- **Button classes** — `.btn`, `.btn-primary`, `.btn-danger`, `.btn-outline`, `.btn-sm`
- **Form styles** — `.form-group`, `.form-row`, `.form-card`, `.form-actions`
- **Card component** — `.card` with shadow and border
- **Table styling** — Clean table with sticky headers
- **Utility classes** — `.badge`, `.loading`, `.error-message`, `.success-message`, `.empty-state`
- **Responsive grid** — `.form-row` goes from 2-column to 1-column on mobile

---

7. Key Design Decisions

DECISION	WHY
SCSS over plain CSS	Variables, nesting, and modular files (one `.scss` per component) keep styles organized and DRY.
BEM-like naming	Prefixing classes with the component name (e.g., `.bean-card__name`, `.bean-card__actions`) prevents style conflicts between components.
Axios in a single service file	All API calls are centralized in `services/api.js`. If the backend URL changes or we add auth tokens, we only edit one file.
Functional components with Hooks	Modern React uses `useState` for local state and `useEffect` for data fetching — no class components, simpler code.
Forms handle both create and edit	`BeanForm` checks if a URL `:id` exists to determine mode. This avoids duplicating the form for add vs edit.
Error + empty state handling	Every page shows user-friendly messages for loading, errors, and empty data — never a blank screen or broken UI.
Vite proxy for development	Avoids CORS issues entirely during development without needing to configure CORS on the backend.